
Upgrading Python CLIs
From Scripts to Interactive Tools
Avik Basu
Staff Data Scientist
Intuit

About Me

- Based in Sunnyvale, CA
- Staff Data Scientist at Intuit
- Editor at PyOpenSci
- Love RPG!
- Driving is therapy! 🕹️

Agenda

1. **Why CLIs?** - Motivation
2. **Argparse** — Baseline functionality
3. **Typer** — Function signatures as CLIs
4. **Rich** — Output that communicates
5. **Textual** — Interactive TUIs
6. **Production patterns** — How to scale things up!

My first CLI experience

- Year **2013**
- Just installed **Ubuntu**
- First **Python project**
- Learning **Git** as a CLI

My first CLI experience

- Year **2013**
- Just Installed **Ubuntu**
- First **Python project**
- Learning **Git** as a CLI

```
➜  cool-project git:(master) ✗ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   requirements.txt
        new file:   src/data.py
        new file:   src/model.py

➜  cool-project git:(master) ✗ git commit
```

I typed `git commit`...

...and suddenly I was

here

I typed `git commit`...

...and suddenly I was

here

```
# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# On branch master
#
# Initial commit
#
# Changes to be committed:
#       new file:   requirements.txt
#       new file:   src/data.py
#       new file:   src/model.py
#
~
~
~
~
~
```

OK, I wrote a message...

...now how do I save and exit?

```
first commit wohoooo!
# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# On branch master
#
# Initial commit
#
# Changes to be committed:
#       new file:   requirements.txt
#       new file:   src/data.py
#       new file:   src/model.py
#
~
~
~
~
~
```

Why great CLIs matter

- Fast
- Composable
- Scriptable
- Universal
- Agent-contract

Our Running Example

A tool to monitor ML model explainability

- Capture explanations as the model runs in production
- Understand which features drive the predictions
- Compare explanations across different model versions or time windows

Three Commands

Command	What it does
`log`	Log explanation values for a batch of predictions
`report`	Generate summary and drift reports (2 sub-commands)
`watch`	Live monitoring dashboard

No ML background needed. Just think of it as:

log data → analyze data → watch data live

What we want from our CLI

1. Distributable in a venv and globally
2. Support Subcommands
3. Type-safe inputs
4. Config hierarchy
5. Progress feedback
6. Fast
7. Good defaults

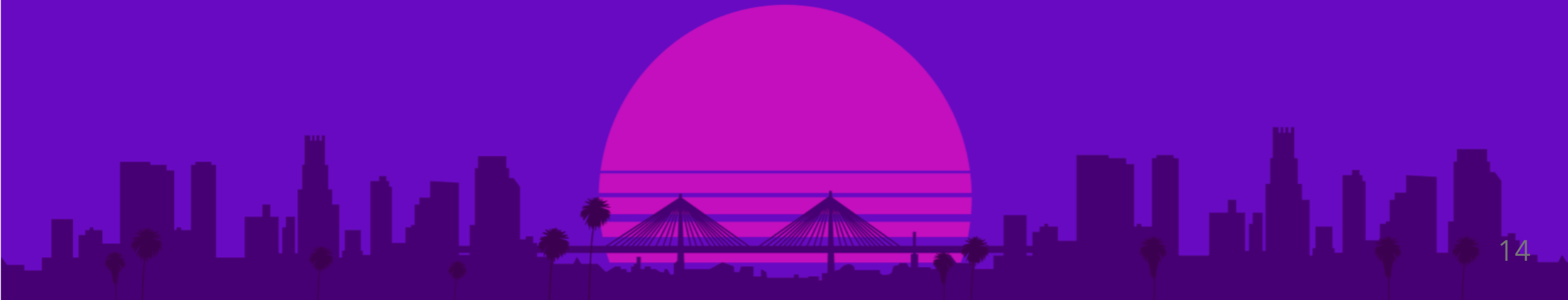
Make it a command

```toml
[project.scripts]
shapmonitor = "shapmonitor.cli:app"
```

- `pip install` puts `shapmonitor` on `$PATH`.
- For local hacking: `python -m shapmonitor` works too if there is a `__main__.py` setup

Argparse

Imperative Style



The log command, with argparse

```python
# shapmonitor/__main__.py
import argparse, os, sys

def main():
    parser = argparse.ArgumentParser(
        prog="shapmonitor",
        description="Log SHAP values for a batch of predictions.",
    )
    parser.add_argument("data", help="Path to CSV file with feature data")
    parser.add_argument("--model", required=True, help="Path to model file")
    parser.add_argument("--data-dir", default="./shap_logs")
    # more args...

    args = parser.parse_args()

    if not os.path.exists(args.data):
        print(f"Error: data file not found: {args.data}", file=sys.stderr)
        sys.exit(1)
    # same for model
    if not 0.0 ⩽ args.sample_rate ⩽ 1.0:
        print("Error: --sample-rate must be between 0.0 and 1.0", file=sys.stderr)
        sys.exit(1)
    # ... and *now* we can read the CSV and compute SHAP values
```

~20 lines of plumbing before we touch the data.

Help command - built-in

```
$ shapmonitor log --help
```

```
usage: shapmonitor log [-h] --model MODEL
[--data-dir DATA_DIR] [--sample-rate SAMPLE_RATE]
[--model-version MODEL_VERSION] data

positional arguments:
  data                  Path to CSV file with feature data

options:
  -h, --help            show this help message and exit
  --model MODEL         Path to pickled model
  --data-dir DATA_DIR   Output directory
  --sample-rate SAMPLE_RATE
                        Sampling rate (0.0-1.0)
  --model-version MODEL_VERSION
                        Model version tag
```

The `log` command

```
$ shapmonitor log demo/batch_current.csv --model demo/model.pkl --data-dir ./logs
```

```
Logged 100 samples to ./logs
```

It writes the explanation (SHAP) values to disk.

The `report summary` command

```
$ shapmonitor report summary --data-dir ./logs
```

```
Feature                Mean |SHAP|       Mean        Std
-------------------------------------------------------
age                          0.1159     0.0106     0.1298
recent_inquiries             0.1053    -0.0134     0.1165
employment_years             0.0806     0.0136     0.1001
credit_score                 0.0563    -0.0075     0.0673
payment_history              0.0476    -0.0178     0.0612
debt_ratio                   0.0340    -0.0187     0.0427
income                       0.0120    -0.0003     0.0183
num_accounts                 0.0054    -0.0001     0.0088
```

It works but, **we can do better**.

The pain points

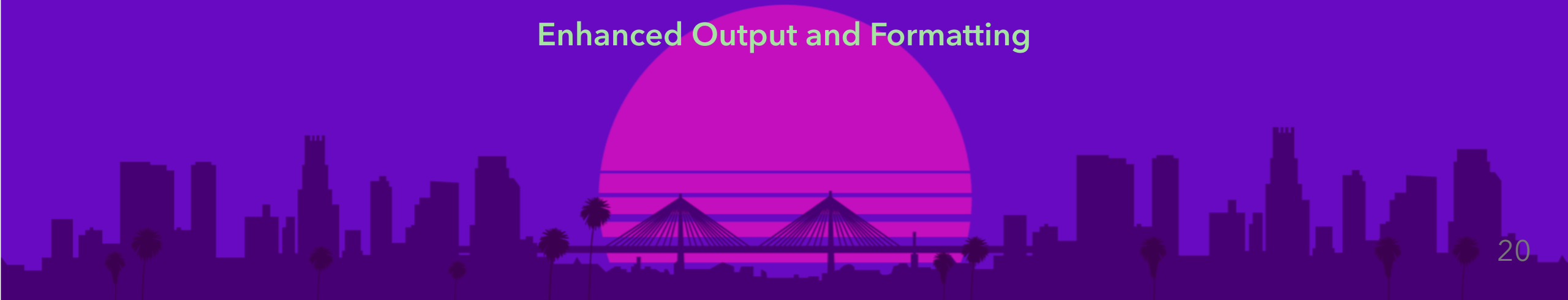
1. Verbose argument definitions
2. Manual validation
3. No Config hierarchy
4. Text-only output

Typer

Core Idea: The Function Signature is the CLI

Rich

Enhanced Output and Formatting



```python
# shapmonitor/cli/log.py
from pathlib import Path
from typing import Annotated
import typer

def log_command(
    data: Annotated[Path, typer.Argument(exists=True, readable=True)],
    model: Annotated[Path, typer.Option("--model", "-m",
        exists=True, readable=True)],
    data_dir: Annotated[Path, typer.Option(
        envvar="SHAPMONITOR_DATA_DIR")] = Path("./shap_logs"),
    sample_rate: Annotated[float, typer.Option(min=0.0, max=1.0)] = 1.0,
) → None:
    """Log SHAP values for a batch of predictions."""
    ...
```

Type hints become **CLI arguments**. Validation is **Declarative.**

Subcommands and composition

```python
# shapmonitor/cli/report.py — a nested Typer group
report_app = typer.Typer(no_args_is_help=True)

@report_app.command("summary")
def summary_command(...): ...

@report_app.command("drift")
def drift_command(...): ...

# shapmonitor/cli/__init__.py — mount the group under "report"
app.add_typer(report_app, name="report")
```

```
→ shapmonitor report summary --top-k 5
→ shapmonitor report drift --ref last-14d..last-7d
```

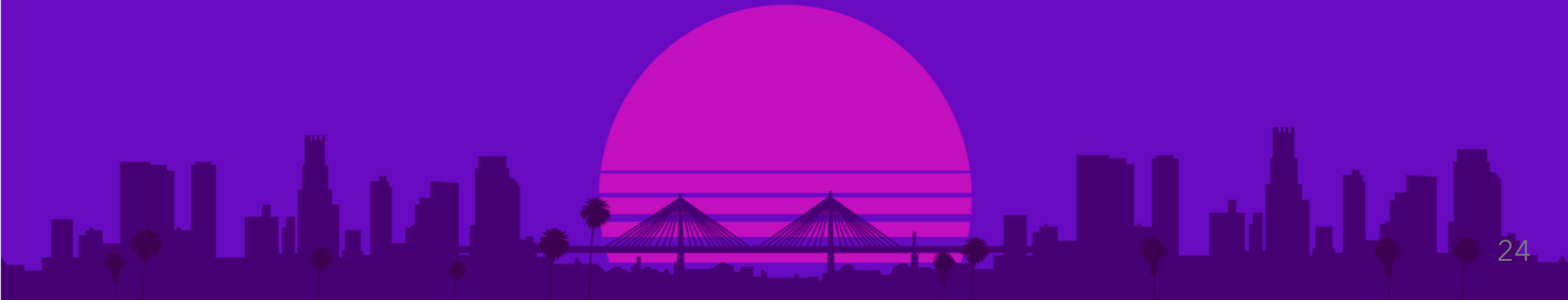
Some Good Typer Defaults

```python
# shapmonitor/cli/__init__.py
app = typer.Typer(
    name="shapmonitor",
    no_args_is_help=True,     # no args? show help, don't error
    help="Monitor SHAP explanations over time.",
    rich_markup_mode="rich",  # styled help output
)
```

- `no_args_is_help=True` – bare `shapmonitor` prints help, not a usage error
- `rich_markup_mode="rich"` – colored, boxed help output from the docstring

Enhanced Output

Rich



Makes life better in a terminal

Rich provides:

- **Progress bars** – feedback for long-running tasks
- **Tables** – structured data that's easy to scan
- **Panels** – grouped information with borders
- **Styled text** – emphasis, color-coded severity
- **Tracebacks** – readable, syntax-highlighted errors

Status Indicators

```python
# shapmonitor/cli/log.py
from rich.progress import Progress, SpinnerColumn, TextColumn

with Progress(
    SpinnerColumn(),
    TextColumn("[progress.description]{task.description}"),
    console=console,
    transient=True,  # disappears when done
) as progress:
    progress.add_task("Computing SHAP values...", total=None)
    monitor.log_batch(X)
```

```
⠋ Computing SHAP values...
```

- **Transient spinners** for indeterminate work.
- **Progress bars** when you know the total.

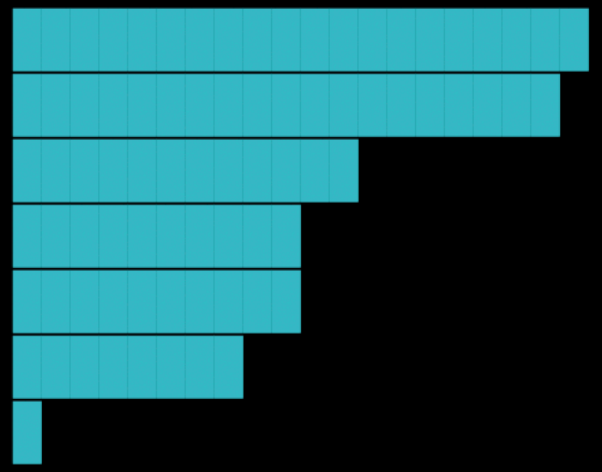
Tables with visual weight

```python
# shapmonitor/cli/report.py — summary_command
from rich.table import Table

table = Table(title=f"SHAP Summary  ({p.start} → {p.end})")
table.add_column("Feature", style="bold")
table.add_column("Mean |SHAP|", justify="right")
table.add_column("", justify="left")       # inline bar
table.add_column("Mean", justify="right")
table.add_column("Std", justify="right")

for feat, row in summary_df.iterrows():
    bar_len = int(20 * row["mean_abs"] / max_mean_abs)
    bar = "[cyan]" + "\u2588" * bar_len + "[/cyan]"
    table.add_row(str(feat), f"{row['mean_abs']:.4f}", bar,
                  f"{row['mean']:.4f}", f"{row['std']:.4f}")

console.print(table)
```

```
(shap-monitor-py3.13) ➜  shap-monitor git:(feat/cli) ✗ shapmonitor report summary
```

Samples: 5200

SHAP Summary (2026-04-11 00:00:00 → 2026-04-18 00:00:00)

Feature	Mean |SHAP|		Mean	Std
recent_inquiries	0.1110		0.0287	0.1180
age	0.1057		0.0015	0.1191
employment_years	0.0689		0.0050	0.0880
payment_history	0.0604		-0.0402	0.0687
debt_ratio	0.0559		-0.0419	0.0605
credit_score	0.0495		0.0014	0.0607
income	0.0077		-0.0012	0.0118
num_accounts	0.0050		-0.0017	0.0076

Styled status and panels

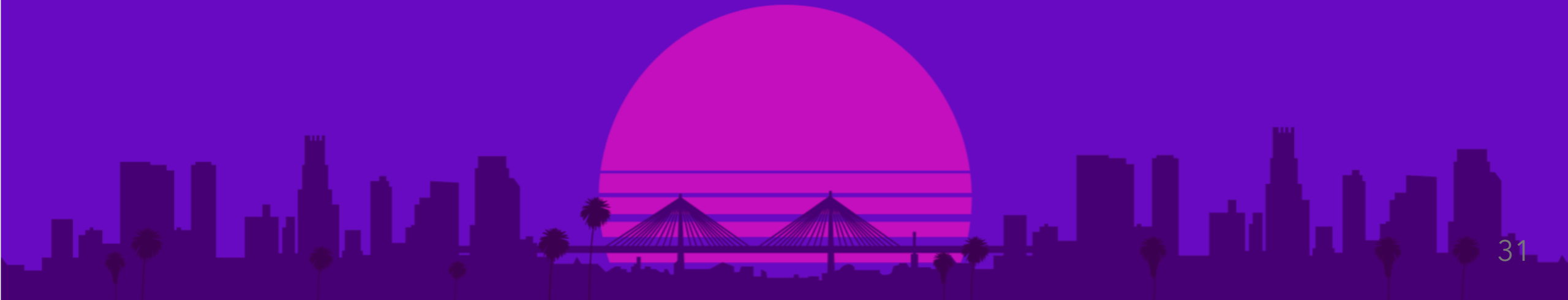
```python
# shapmonitor/cli/report.py — drift_command
from rich.panel import Panel

alerts = int((drift_df["psi"] ⩾ 0.25).sum())
warnings = int(((drift_df["psi"] ⩾ 0.1) & (drift_df["psi"] < 0.25)).sum())
stable = int((drift_df["psi"] < 0.1).sum())

headline = " · ".join([
    f"[red]{alerts} alert{'s' if alerts ≠ 1 else ''}[/red]",
    f"[yellow]{warnings} warning{'s' if warnings ≠ 1 else ''}[/yellow]",
    f"[green]{stable} stable[/green]",
])

console.print(Panel(headline, title="Drift Report", border_style="blue"))
```

```
(shap-monitor-py3.13) ➜  shap-monitor git:(feat/cli) ✗ shapmonitor report drift \
[--data-dir demo/shap_logs --ref last-14d..last-7d --curr last-7d..now          ]
╭─ Drift Report  (ref: 2026-04-04 00:00:00→2026-04-11 00:00:00  vs  curr: 2026-04-11 ─╮
│ 0 alerts · 2 warnings · 6 stable                                                    │
```

Feature	PSI	Status	|SHAP| ref	|SHAP| curr	Δ |SHAP|	Rank Δ
payment_history	0.2356	warning	0.0406	0.0539	0.0134	—
credit_score	0.1010	warning	0.0580	0.0553	-0.0027	—
employment_years	0.0832	stable	0.0791	0.0910	0.0118	—
num_accounts	0.0498	stable	0.0059	0.0043	-0.0016	—
debt_ratio	0.0414	stable	0.0343	0.0336	-0.0006	—
recent_inquiries	0.0294	stable	0.1041	0.0990	-0.0051	—
age	0.0205	stable	0.1154	0.1178	0.0024	—
income	0.0085	stable	0.0127	0.0130	0.0003	—

Interactive TUI

Textual



When static output isn't enough

Some tasks need **interactivity**:

- **Live monitoring** — data that updates in real time
- **Exploration** — filter, sort, navigate large datasets
- **Keyboard-driven workflows** — no mouse, no re-running

Textual gives you a full application framework in the terminal: widgets, layouts, events, CSS styling, keybindings.

App structure

```python
# shapmonitor/cli/_watch_app.py
from textual.app import App, ComposeResult
from textual.widgets import DataTable, Footer, Header, Input

class WatchApp(App):
    TITLE = "shapmonitor watch"
    CSS = """
    #filter-input { dock: top; margin: 0 1; height: 3; }
    DataTable { height: 1fr; }
    """
    def compose(self) -> ComposeResult:
        yield Header(show_clock=True)
        yield Input(placeholder="Type to filter features...", id="filter-input")
        yield DataTable(id="shap-table")
        yield Footer()
```

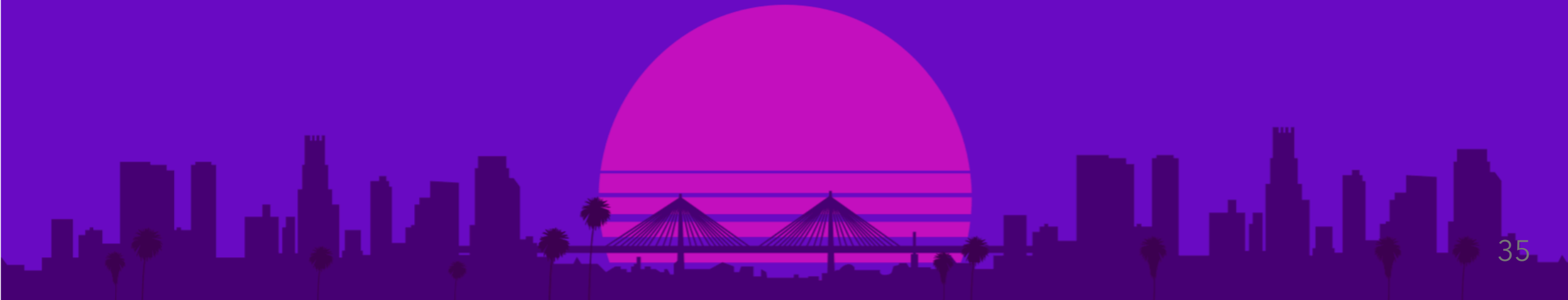
- `compose()` defines the layout.
- CSS controls sizing.

Live data with auto-refresh

```python
def on_mount(self) -> None:
    table = self.query_one(DataTable)
    table.add_columns("Feature", "Mean |SHAP|", "Std", "PSI", "Status")

    self._load_data()  # initial load
    self.set_interval(self._refresh_interval, self._load_data)

def _load_data(self) -> None:
    analyzer = SHAPAnalyzer(get_backend(self._data_dir))
    period = parse_period(self._period_spec)
    self._summary = analyzer.summary(period.start, period.end)
    self._refresh_table()
```

The table rebuilds with fresh data every 5 seconds.

Production Patterns



Composability – pipeable output

A great CLI plays well with `jq`, `xargs`, **and** humans at the same time.

```python
if json_mode:
    typer.echo(json.dumps(data, default=str))
    return
render_rich_table(data)
```

```
$ shapmonitor report summary --json | jq '.features[:1]'
```


```
[
  {
    "feature": "recent_inquiries",
    "mean_abs": 0.11099565029144287,
    "mean": 0.028652228415012360,
    "std": 0.11797577887773514,
    "min": -0.2615908980369568,
    "max": 0.2352857291698455 8
  },
]
```

Imports are EXPENSIVE for a CLI application

```python
# shapmonitor/__init__.py
import logging
from shapmonitor.monitor import SHAPMonitor

logging.getLogger(__name__).addHandler(logging.NullHandler())

__all__ = ["SHAPMonitor"]

# 1. `import shapmonitor`
# 2. `from shapmonitor import shapmonitor`
# Both of them basically take the same time.
```

The cost: every `import` of the package pays for a lot of dependencies; even when the caller never touches the main module.

For a CLI, that means even `shapmonitor --help` can become slow.

Lazy Import Strategies

- Use local imports, e.g. inside a function
- Use the `lazy-loader` library
- Use `TYPE_CHECKING` for expensive type hints
- Wait for Python 3.15 (PEP 810)

Lazy module attributes (Thanks to: PEP 562 🙏)

```python
# shapmonitor/__init__.py
__all__ = ["SHAPMonitor"]

def __getattr__(name):
    if name == "SHAPMonitor":
        from shapmonitor.monitor import SHAPMonitor
        return SHAPMonitor
    raise AttributeError(f"module {__name__!r} has no attribute {name!r}")
```

- `import shapmonitor` → loads only the module shell
- `from shapmonitor import SHAPMonitor` → triggers the dependency loading **only on first access**

Distribution

Not everyone is a python developer or a python user! 🙂

```
pip install shap-monitor              # library users
```

```
pipx install "shap-monitor[cli]"      # CLI users installed globally
```

```
uvx install "shap-monitor[cli]"       # CLI users installed globally
```

```
brew install shap-monitor             # if needed
```

Config hierarchy

Flags → env vars → defaults - the precedence users expect.

```python
data_dir: Annotated[
    Path,
    typer.Option(
        envvar="SHAPMONITOR_DATA_DIR",   # ← falls back to env var
    ),
] = Path("./shap_logs")                  # ← falls back to default
```

```
# All three work, in priority order:
$ shapmonitor log data.csv --data-dir /custom/path
$ SHAPMONITOR_DATA_DIR=/custom/path shapmonitor log data.csv
$ shapmonitor log data.csv  # uses ./shap_logs
```

One line gives us 3 config layers.

Where should your CLI's files live?

A proper CLI can have **three categories** of files:

- **Config** – settings the user edits
- **Data** – persistent state your tool writes
- **Cache** – regeneratable files

The **XDG Base Directory Spec** – Linux's standard home for each:

Kind	Env var	Default
config	`$XDG_CONFIG_HOME`	`~/.config`
data	`$XDG_DATA_HOME`	`~/.local/share`
cache	`$XDG_CACHE_HOME`	`~/.cache`

Spec: specifications.freedesktop.org/basedir/latest

Cross-platform with `platformdirs`

```python
from platformdirs import user_config_dir, user_data_dir, user_cache_dir

config_dir = Path(user_config_dir("your_app"))
data_dir   = Path(user_data_dir("your_app"))
cache_dir  = Path(user_cache_dir("your_app"))

# Linux:   ~/.config/yourapp, ~/.local/share/yourapp, ~/.cache/yourapp
# macOS:   ~/Library/Preferences/yourapp, ~/Library/Application Support/yourapp, ...
# Windows: %APPDATA%\yourapp, %LOCALAPPDATA%\yourapp, ...
```

Notable CLIs using this: `black`, `poetry`, `tox`.

When to reach for what

You need...	Use...	Complexity
Plain single script	**Argparse**	Low
Just argument parsing	**Typer**	Low
Readable, structured output	+ **Rich**	Low
Progress bars, spinners	+ **Rich**	Low
Live data, interactivity	+ **Textual**	Medium
Full TUI application	+ **Textual**	Higher

References

Typer — CLIs from type hints

Docs: typer.tiangolo.com · Source: github.com/tiangolo/typer

Rich — Beautiful terminal output

Docs: rich.readthedocs.io · Source: github.com/Textualize/rich

Textual — Full TUI application framework

Docs: textual.textualize.io · Source: github.com/Textualize/textual

Thank you! 🙏

Questions?

LinkedIn

Slides

shap-monitor